

Лабораторна робота 5

Кластеризація даних засобами Python

//бібліотеки numpy, scikit-learn, pandas, matplotlib

Завдання на лабораторну роботу

1. Розглянути наведені в теоретичній відомостях методи і функції кластерного аналізу даних. За допомогою вбудованої довідки вивчити параметри функцій `hclust()` і `kmeans()`.
2. Для власного набору даних провести розрахунок парних відстаней між об'єктами та кластерний аналіз, використовуючи метод метод k-середніх (побудувати необхідні залежності, проаналізувати отримані результати).
3. Для масиву даних, що містяться в файлі `stocks.csv`, сформувати та візуалізувати ієрархічне дерево кластерів (дендрограму) за допомогою різних алгоритмів, використовуючи відповідні функції пакету R-studio. Провести аналіз отриманих результатів.
4. Проведіть кластерний аналіз набору даних з сайту <http://data.worldbank.org>. Виконайте кластеризацію країн за рівнем безробіття використовуючи дані з файлу `sl.uem.totl.zs_Indicator_en_csv_v2.csv`.

Для початку потрібно імпортувати чотири бібліотеки `numpy`, `pandas`, `Scikit-learn`, `matplotlib`.

- `Numpy` - це бібліотека для обчислень з підтримкою високорівневих обчислень
- `Pandas` – це бібліотека для роботи з даними на Python, що надає інструменти для читання та запису даних у різних форматах, включаючи CSV, Excel і багато інших
- `Scikit-learn` – це бібліотека для машинного навчання в Python, що надає інструменти для класифікації, регресії, кластеризації, зменшення розмірності і багато іншого
- `Matplotlib` - бібліотека для візуалізації даних в Python, що надає інструменти для створення різних типів графіків, включаючи лінійні графіки, стовпчасті графіки, кругові діаграми, гістограми та інші.

```
import numpy as np
import pandas as pd
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
```

Дальше згенеруємо випадкові дані про фрукти

```
np.random.seed(0)
fruits = ['яблуко', 'груша', 'виноград', 'апельсин', 'ананас',
          'манго']
weights = np.random.randint(50, 200, size=100)
prices = np.random.randint(1, 10, size=100) + np.random.rand(100)
```

Задаємо список фруктів, випадкові ваги та ціни для 100 фруктів, та створюємо `DataFrame`, що містить дані про фрукти.

```
fruits_data = pd.DataFrame({'Фрукт': np.random.choice(fruits,
size=100),
                           'Вага': weights,
                           'Ціна': prices})
```

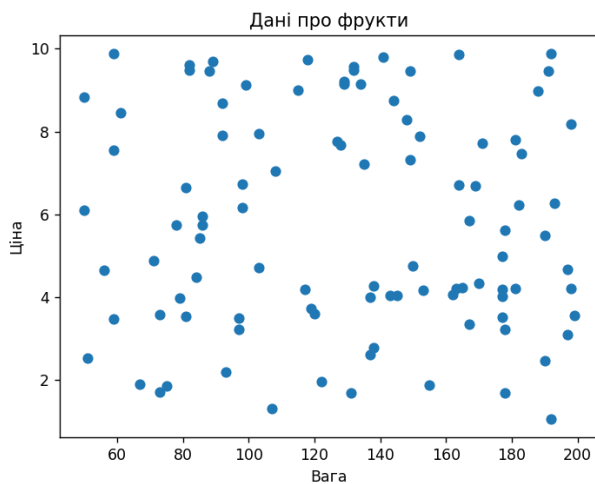
Виводимо перші 10 рядків даних про фрукти.

```
print(fruits_data.head(10))
```

	Фрукт	Вага	Ціна
0	груша	97	3.489549
1	ананас	167	3.338985
2	груша	117	4.179490
3	виноград	153	4.170987
4	ананас	59	3.463451
5	яблуко	71	4.874573
6	яблуко	86	5.944120
7	виноград	137	2.608253
8	манго	120	3.596655
9	яблуко	138	2.783644

Візуалізуємо дані про фрукти на графіку, де вага фруктів - вісь x, а ціна - вісь y.

```
plt.scatter(fruits_data['Вага'], fruits_data['Ціна'])
plt.xlabel('Вага')
plt.ylabel('Ціна')
plt.title('Дані про фрукти')
plt.show()
```



Використовуємо алгоритм k-середніх для кластеризації даних про фрукти в 3 кластери.

```
kmeans = KMeans(n_clusters=3, random_state=0)
X = fruits_data[['Вага', 'Ціна']]
kmeans.fit(X)
```

Додаємо мітку кластеру до кожного рядка даних про фрукти.

```
fruits_data['Кластер'] = kmeans.labels_
```

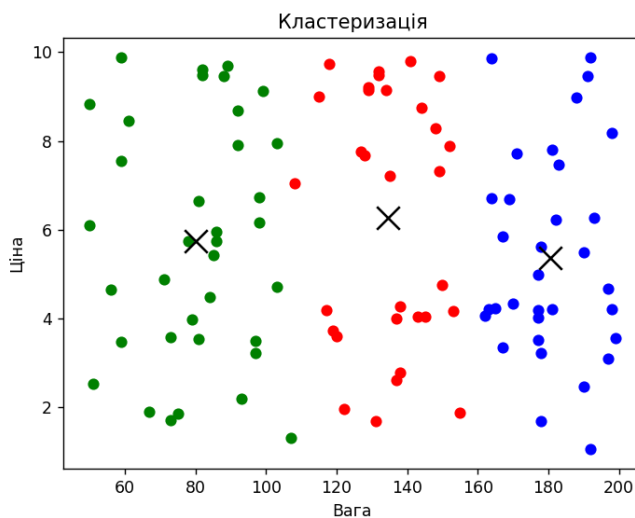
Виводимо перші 10 рядків даних про фрукти, що містять мітки кластерів.

```
print(fruits_data.head(10))
```

	Фрукт	Вага	Ціна	Кластер
0	груша	97	3.489549	1
1	ананас	167	3.338985	2
2	груша	117	4.179490	0
3	виноград	153	4.170987	0
4	ананас	59	3.463451	1
5	яблуко	71	4.874573	1
6	яблуко	86	5.944120	1
7	виноград	137	2.608253	0
8	манго	120	3.596655	0
9	яблуко	138	2.783644	0

Візуалізуємо кластери на графіку, де кожен кластер має свій колір, а центроїди кластерів маркером 'x'. Графік показує залежність між вагою та ціною фруктів, а кластери показують, які фрукти схожі між собою за цими параметрами.

```
colors = ['r', 'g', 'b']
for i in range(3):
    cluster_data = fruits_data[fruits_data['Кластер'] == i]
    plt.scatter(cluster_data['Вага'], cluster_data['Ціна'],
c=colors[i])
plt.scatter(kmeans.cluster_centers_[0],
kmeans.cluster_centers_[1], marker='x', c='k', s=200,
label='Центроїди')
plt.xlabel('Вага')
plt.ylabel('Ціна')
plt.title('Кластеризація')
plt.show()
```



Завдання 3

В коді нижче ми імпортуємо необхідні бібліотеки після чого завантажуюмо набір даних `stock.csv` і вибираємо з нього 50 перших рядків. Отримуємо колонки з датами і видаляємо їх з `dataframe` після чого робимо кластеризацію за допомогою методу `ward` та метрики `Euclidian` після чого за допомогою бібліотеки `matplotlib` графічно будуюмо дендрограму

```
import pandas as pd
import matplotlib.pyplot as plt
from scipy.cluster.hierarchy import linkage, dendrogram

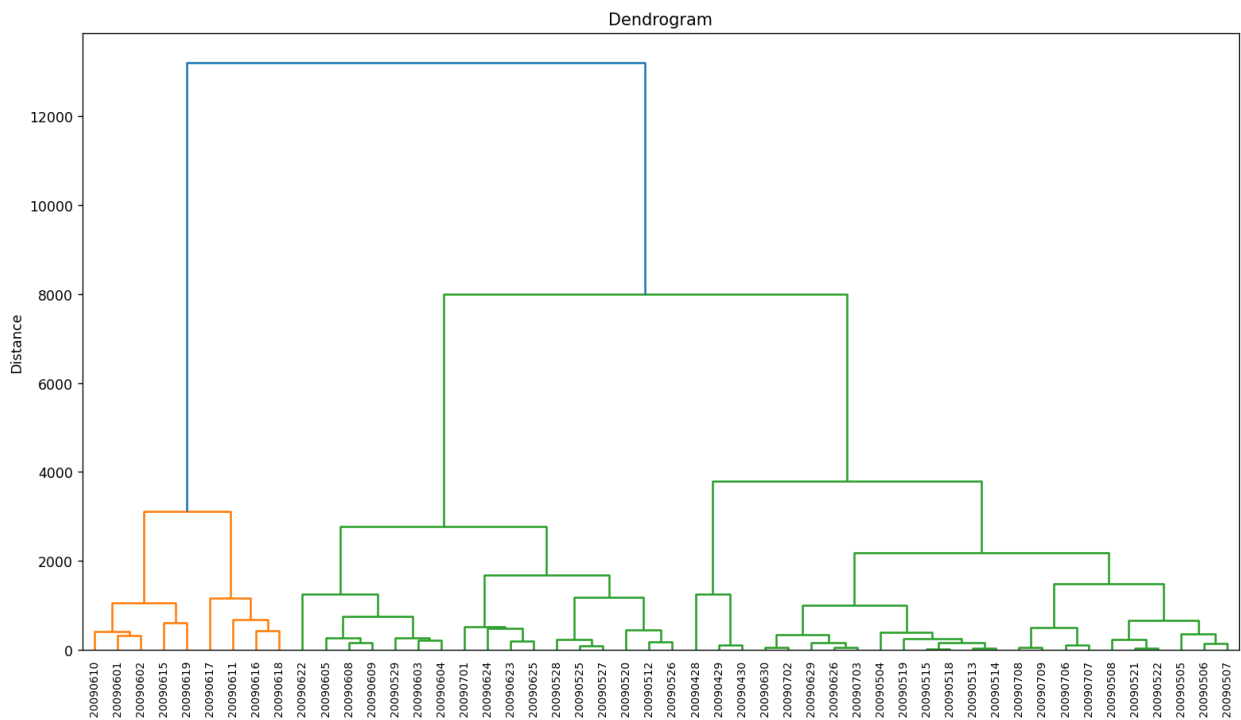
df = pd.read_csv('stock.csv').head(50)

dates = df['DATE']

data = df.drop('DATE', axis=1)

x = linkage(data, method='ward', metric='euclidean')

plt.figure(figsize=(15, 10))
plt.title('Dendrogram')
plt.xlabel('Stocks')
plt.ylabel('Distance')
dendrogram(x, labels=dates.tolist(), leaf_rotation=90)
plt.show()
```



У цьому прикладі ми використовуємо метод `'ward'` для об'єднання кластерів та евклідову метрику для вимірювання відстаней між об'єктами. Візуалізуємо дендрограму, де по осі `X` відображаються дати, а по осі `Y` - відстані між кластерами. Можна спостерігати, як об'єкти об'єднуються в кластери на різних рівнях відповідно до відстаней між ними. Аналізуючи отриману дендрограму,

можна робити висновки про рівень схожості між різними акціями та можливості їх об'єднання в кластери на основі рівня подібності.

Завдання 4

Цей код виконує кластерний аналіз за допомогою алгоритму k-середніх на даних про безробіття в різних країнах.

1. Спочатку, імпортуються необхідні бібліотеки Pandas, NumPy, Matplotlib, Scikit-learn.
2. Наступним кроком є завантаження даних з CSV-файлу "somedata.csv", обмеження до перших 50 рядків та вибір потрібних стовпців ("Country Name" та "2019") за допомогою методу read_csv() та індексування.
3. Далі, відбувається видалення рядків з пропущеними значеннями методом dropna().
4. Змінюємо індекс датафрейму на назви країн методом set_index().
5. Наступним кроком є стандартизація даних за допомогою класу StandardScaler().
6. Використовуючи алгоритм k-середніх, виконується кластеризація даних на 3 кластери. Для цього створюється об'єкт KMeans та викликається метод fit().
7. Остаточно, змінюємо датафрейм додавши до нього ознаку кластера, до якого кожна країна відноситься, та візуалізуємо результати на графіку методом scatter(). Це дає змогу зрозуміти, які країни належать до якого кластера, з урахуванням їх рівня безробіття.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

data = pd.read_csv('somedata.csv').head(50)

data = data[['Country Name', '2019']]

data = data.dropna()

data = data.set_index('Country Name')

print(data.head())

scaler = StandardScaler()
scaled_data = scaler.fit_transform(data)

kmeans = KMeans(n_clusters=3, random_state=0).fit(scaled_data)

data['Cluster'] = kmeans.labels_
```

```
plt.scatter(data['2019'], data.index, c=data['Cluster'],
cmap='viridis')
plt.xlabel('Unemployment Rate (%)')
plt.ylabel('Country')
plt.title('Кластеризація за рівнем безробіття')
plt.show()
```

